

LabGuardian 项目报告

基于边缘 AI 的智能实验助教系统

——后端代码、前端代码与项目方案综合分析报告

项目名称	LabGuardian: 基于边缘 AI 的智能实验助教系统
代码仓库	LabGuardian-Server; LabGuardian-Web
报告用途	课程/竞赛项目报告、阶段验收材料、答辩材料基础稿
撰写依据	项目方案 PDF、后端仓库 README/docs/app 代码、 前端仓库 README/src 代码
版本说明	以当前公开 main 分支代码和文档为依据; 未包含真实硬件实测数据
日期	2026 年

说明: 本报告可根据学校/团队信息继续补充封面要素。

目录

摘要

第一章 项目概述

第二章 项目背景与需求分析

第三章 总体设计方案

第四章 后端系统设计与实现

第五章 前端系统设计与实现

第六章 数据结构与接口设计

第七章 关键技术与创新点

第八章 测试验证与质量保障

第九章 部署运行与边缘化方案

第十章 项目进度、现状与差异说明

第十一章 风险分析与改进计划

第十二章 应用价值、推广路径与实施组织

第十三章 结论

参考资料

附录：术语与文件索引

注：本目录为静态目录，用于提交和阅读。若需要自动页码，可在 Word 中插入或更新自动目录。

摘要

LabGuardian 是一套面向高校电子类基础实验的智能实验助教系统。项目以面包板电路搭建、元器件引脚定位、孔位映射、拓扑重构、参考电路比对和诊断解释为主线，试图解决传统电子实验教学中教师巡检压力大、学生排错效率低、真实电路遮挡严重、原理图与实物连接映射困难等问题。根据项目方案，系统面向 DK-2500 等边缘计算平台，强调本地化推理、知识检索、异构计算和教学数据隐私保护；而从当前代码仓库看，项目已形成较完整的后端 Pipeline、结构化网表、逻辑比较、PCM Agent、Web 前端演示和硬件遥测基础。

本报告在项目方案 PDF 的背景分析基础上，结合 LabGuardian-Server 与 LabGuardian-Web 两个仓库的 README、docs 和核心源码，对项目目标、需求、总体架构、后端实现、前端实现、接口数据结构、关键技术、测试部署方案、项目进度和后续改进方向进行系统梳理。报告特别区分“初选方案规划内容”和“当前仓库已实现内容”：当前主链路聚焦面包板电路的结构化诊断，采用 S1 组件检测、S1.5 全图引脚姿态检测、S2 孔位映射、S3 拓扑与 netlist_v2 生成、S4 逻辑参考比较、S5 语义分析的事实链；PCM Agent 以结构化证据和白盒工具为基础生成解释，而不是把事实判断交给大模型。PCB AOI、端侧 VLM 微观缺陷识别等能力更适合作为后续扩展方向纳入路线图。

关键词：边缘 AI；电子实验教学；面包板电路；YOLO-Pose；BoardSchema；netlist_v2；图同构比较；PCM Agent；RAG 知识增强；React 前端。

第一章 项目概述

1.1 项目定位

LabGuardian 的项目定位是“面向电子实验教学场景的边缘智能助教系统”。与通用电路仿真软件不同，它不是只在虚拟环境中验证电路原理，而是从学生真实搭建的面板照片出发，将图像中的元件、引脚、孔位和电气连接转换为可验证、可审计的结构化事实，再与教师给出的参考电路进行逻辑比较，最后向学生输出错误位置、错误原因、风险等级和修改建议。系统的核心价值在于把实验教学中的经验性巡检过程数字化、结构化和可解释化，使教师能够从重复排错中解放出来，学生也能在搭建过程中获得及时反馈。

从代码仓库的当前状态看，LabGuardian-Server 已经将主线收束为“`component_id + pin_name + hole_id → electrical_node_id → electrical_net_id → netlist_v2`”的事实链。这个事实链非常关键，因为它把视觉识别结果与电气语义连接起来：`component_id` 表示识别出的元件实例，`pin_name` 表示元件具体引脚，`hole_id` 表示物理面包板孔位，`electrical_node_id` 表示面包板静态导通节点，`electrical_net_id` 表示由导线和元件连接合并后的电气网络。只要这条链路可靠，系统就能把“图片中看起来接在某处”的模糊观察转化为“该引脚属于哪个电气网络”的确定性判断。

1.2 建设目标

- 构建从图像上传、元件识别、引脚检测、孔位映射、拓扑重构到诊断解释的完整演示链路。
- 形成稳定的数据契约，使前端、后端、Agent、知识库和测试用例都围绕同一套结构化事实运行。
- 面向电子实验教学输出可解释结果，包括错误码、证据引用、建议动作、风险等级和前端高亮目标。
- 为 DK-2500 等边缘设备部署预留模型路径、运行元数据、硬件遥测和后续 OpenVINO/INT8 优化入口。
- 在系统架构上坚持“事实判断由确定性算法完成，Agent 负责解释与引导”，降低大模型幻觉风险。

1.3 报告范围与依据

本报告覆盖项目背景、需求分析、总体方案、后端实现、前端实现、接口和数据结构、关键技术、测试部署、进度现状和改进计划。资料来源包括初选项目设计方案书、LabGuardian-Server 仓库 README 与 docs 文档、LabGuardian-Server 的核心代码文件，以及 LabGuardian-Web 仓库 README、package.json、API 封装和 demo 页面源码。报告不是简单复述初选方案，而是结合代码实现对项目进行工程化整理。

需要说明的是，初选方案中包含 PCB 无监督 AOI、Anomalib 热力图和 NPU VLM 微观诊断等扩展设想；但当前 Server README 明确提示 PCB/AOI 相关代码已从仓库移除，当前 Agent 图中也删除了 `vlm_explain` 分支。因此，本报告将这些内容放在“后续拓展与风险控制”部分，而不把它们描述为当前主链路已完成能力。

第二章 项目背景与需求分析

2.1 教学背景

高校电子类基础实验通常覆盖电路分析、模拟电子技术、数字电子技术、电子工艺实训等课程，是学生从理论学习走向工程实践的重要环节。传统课堂中，学生需要依据原理图在面包板上插接电阻、电容、二极管、LED、晶体管、运放或其他集成芯片，并通过电源、示波器、万用表等仪器观察实验现象。这个过程本质上需要学生完成三个映射：从抽象电路原理映射到元件功能，从二维原理图映射到面包板孔位，从实验现象映射到具体故障原因。对初学者而言，任何一个映射失败都可能导致实验停滞。

项目方案指出，电子实验教学普遍存在较高的师生比，教师难以在同一时间覆盖所有实验台。学生遇到问题后，往往需要排队等待教师检查；教师到场后也需要从“元件是否插对、引脚是否接反、孔位是否同一导通组、电源轨是否误接、导线是否短路”等基础问题逐项排查。这类排查高度依赖经验，重复性强，但又不能简单省略，因为极性反接和电源短路会造成芯片损坏甚至实验安全风险。

2.2 核心痛点

第一，真实面包板图像的遮挡问题明显。面包板上导线常常互相交叠，元件本体可能遮住引脚，拍摄角度、反光、阴影和焦距都会影响视觉识别。传统目标检测框只能告诉系统“这里可能有一个元件”，但难以准确给出引脚插入了哪个孔位。对于电子诊断而言，元件框级别的识别远远不够，系统必须进入引脚级和孔位级。

第二，学生对面包板静态导通关系不熟悉。面包板主区域通常按行导通，左右半区又被中缝隔开，电源轨还可能存在分段。学生可能认为两个孔在图像上很近就等价，或者忽视 A-E 与 F-J 的隔离关系。系统如果能把 `hole_id` 映射为 `electrical_node_id`，就可以明确告诉学生“当前位置虽然相邻，但不在同一导通节点”或“这两个引脚实际上落在了同一网络，存在短路风险”。

第三，教师需要可解释证据而不是黑盒结论。实验教学场景中的 AI 诊断不能只输出“电路错误”四个字，而应说明错误码、涉及的元件、引脚、孔位、参考连接、当前连接以及建议动作。LabGuardian 当前的 `validator_report_v2` 就围绕这一点设计，它在每条诊断项中保留 `evidence_refs`，并且可被前端转换为框选元件、点亮引脚、标注孔位的高亮协议。

第四，系统需要适应边缘环境。高校实验室网络条件并不总是稳定，课堂图像和学生实验数据也不宜全部上传云端。项目选择边缘设备作为主要运行平台，既能降低依赖外部网络的风险，也方便现场竞赛展示。代码中的 `edge deployment` 文档已经统一模型根目录、默认推理尺寸和 `runtime_metadata`，为后续 DK-2500 真机部署和性能评测提供基础。

2.3 用户角色需求

角色	主要需求	系统响应
学生	快速知道电路哪里错、为什么错、如何改；能上传图片并看到可视化定位。	前端上传图片，显示识别框、孔位、网表、诊断卡片和 Agent 建议。

教师/助教	了解多个实验台状态，定位共性错误，减少重复巡检。	后端维护 ClassroomState，记录工位风险、诊断、网表和缩略图，支持教师看板扩展。
项目开发者	保持视觉、拓扑、比较、Agent 的数据契约稳定，便于协作开发。	README、docs、tests 明确各阶段协议和维护规则，新增逻辑需同步测试和文档。
竞赛评委/验收者	希望看到端到端链路、异构边缘价值、可解释诊断和工程完整性。	演示主线展示从图像到事实链再到诊断解释，并通过 runtime_metadata/telemetry 支撑可观测性。

2.4 功能需求

- 图像接入：支持上传 1 至 3 张 base64 编码的面板图片，当前前端演示默认上传一张主视角图片，后端协议可扩展多视角。
- 组件检测：识别电阻、电容、导线、LED、二极管、三极管、电位器、IC 等教学常见元件，生成全局 component_id。
- 引脚检测：通过全图 YOLO-Pose 或几何规则输出 ordered pins，并保留多视图观测、置信度和来源信息。
- 孔位映射：将引脚关键点吸附到面板 hole_id，并进一步映射到 electrical_node_id。
- 拓扑重构：基于 BoardSchema 和 CircuitAnalyzer 合并导线网络，输出 netlist_v2、拓扑图和 SPICE 网表。
- 参考比较：将当前 netlist_v2 与 logical_reference_v1 参考电路转换为图模型，执行拓扑匹配、角色推断和差异报告。
- 诊断解释：把 validator_report_v2、RuntimeEvidence、ContextPack 和工具结果转化为学生可理解的自然语言建议。
- 人工修正：前端支持端口标注、孔位修正、网络角色指定、IC 标注和引脚极性选择，提交后重跑 S3/S4。
- 可视化：前端显示阶段进度、元件框、引脚点、孔位高亮、网表表格、诊断项和原始 JSON。
- 运行观测：后端支持 runtime_metadata，并提供 CPU、内存、iGPU、NPU 相关硬件遥测协议。

2.5 非功能需求

非功能需求主要包括准确性、可解释性、低延迟、弱网可用、隐私保护、可维护性和可扩展性。准确性体现在视觉识别、孔位吸附、导通节点映射和逻辑比较的综合正确率；可解释性体现在每个错误都有证据引用和可视化目标；低延迟要求端到端推理在课堂交互中足够及时；弱网和隐私要求系统尽量本地运行；可维护性要求不同阶段协议清晰、测试覆盖充分；可扩展性要求新增板型、元件、参考电路、故障案例和教学知识时不需要重写核心链路。

第三章 总体设计方案

3.1 总体架构

LabGuardian 采用前后端分离架构。前端 LabGuardian-Web 负责演示界面、图片上传、参数选择、参考电路选择、结果可视化、人工修正和 Agent 对话；后端 LabGuardian-Server 负责 API 接入、图像处理、模型推理、孔位映射、拓扑构建、逻辑比较、课堂状态、知识检索、Agent 编排和遥测推流。整体架构可以概括为“Web 交互层—API 服务层—Pipeline 事实层—Domain 规则层—Agent/Knowledge 解释层—Infra 支撑层”。

后端 README 的架构图将系统划分为 FastAPI/WebSocket API、Services、Agent/Knowledge、Domain、Pipeline 和 Infra 六层。API 层只做协议入口，不承担领域推理；Services 层负责编排、审计、下发和任务管理；Domain 层放置 BoardSchema、CircuitAnalyzer、validator、risk、reference DSL 等稳定规则；Pipeline 层只输出结构化事实，不直接生成教学话术；Agent/Knowledge 层在事实基础上组织上下文、工具调用和回答；Infra 层包含 Redis、Celery、Docker Compose 和测试夹具。

层级	主要模块	职责说明
Web 交互层	React、Vite、TypeScript、DemoPage、ResultCanvas、NetlistView、AgentChat	完成图片上传、参数配置、结果展示、人工修正、Agent 交互和演示流程编排。
API 层	pipeline.py、angnt.py、classroom.py、websocket.py、telemetry_ws.py	提供同步/异步 Pipeline、Agent 问答、课堂状态、遥测推流等接口。
服务层	pipeline_service、guidance_service、classroom_state、reference_service、agent_service	负责业务编排、结果封装、参考电路解析、课堂态同步和任务状态管理。
Pipeline 层	orchestrator、s1_detect、s1b_pin_detect、s2_mapping、s3_topology、s4_validate、s5_semantic_analysis	从图像输入到诊断事实输出的主链路。
Domain 层	board_schema、circuit、netlist_models、logical_reference、compare、risk、dsl	提供板型、电气网络、参考电路、图比较和风险分类等确定性规则。
Agent/Knowledge 层	RuntimeEvidence、ContextPack、tools、LangGraph、verification、datasheet/circuit/fault KB	把结构化事实转化为可控上下文和教学解释。

3.2 主业务流程

一次完整诊断流程从前端上传图片开始。用户选择参考电路、设定置信度、IoU、推理尺寸和电源轨角色后，前端将图片转为纯 base64 字符串，提交到 POST /api/v1/pipeline/run。后端 PipelineService 解析 reference_id 或 inline reference_circuit，调用 orchestrator 执行 S1 到 S5。执行完成后，服务层将结果封装为 PipelineResult，同时同步到 ClassroomState。前端接收结果后显示识别事实链和诊断报

告，并自动向 Agent 提交“根据当前诊断结果给出演示用诊断解释和下一步建议”的提示，Agent 再基于 ClassroomState 中的 RuntimeEvidence 生成自然语言解释。

如果学生或教师发现视觉映射结果不准确，前端可以进入网表模式进行人工修正。用户可以修正引脚孔位，给输入/输出端口打标，设置网络角色，或补充三极管和 IC 的人工标注。前端随后调用 /api/v1/pipeline/recompute-corrected，把 mapping components 与修正 patch 提交给后端。后端不再重跑视觉检测，而是从修正后的 components 重新执行 S3 拓扑、S4 验证和 S5 语义分析。这种设计兼顾 AI 自动识别与教学现场的人机协同，避免视觉阶段偶发错误导致整个系统不可用。

3.3 技术选型

类别	选型	理由
后端框架	FastAPI、Pydantic、Uvicorn	接口开发效率高，类型契约清晰，适合返回结构化 JSON。
异步任务	Celery、Redis	支持较长模型推理任务异步提交与状态查询，演示阶段也保留同步接口。
视觉算法	Ultralytics YOLO-Detect、YOLO-Pose、OpenCV	覆盖元件检测和引脚关键点检测，便于训练和部署。
拓扑建模	NetworkX、Union-Find、BoardSchema	适合构建二分图、合并导通网络、输出可解释网表。
知识/Agent	PCM ContextPack、deterministic tools、LangGraph、Datasheet/Circuit KB	把事实、工具、回答分层，降低大模型幻觉风险。
前端	React 19、Vite、TypeScript、lucide-react	适合构建交互式单页应用和演示看板。
边缘部署	Docker、OpenVINO/GenAI 预留、runtime_metadata、telemetry	为 DK-2500 本地推理和性能评测做准备。

3.4 设计原则

系统设计遵循四项原则。第一，事实链优先。视觉模型只产生候选事实，最终教学结论必须经过 BoardSchema、拓扑分析和 validator 约束。第二，接口契约优先。S1、S1.5、S2、netlist_v2、validator_report_v2、RuntimeEvidence 和 ContextPack 都以结构化 schema 作为协作边界，避免前后端或 Agent 私自猜测。第三，人机协同优先。系统允许人工端口标注和孔位修正，教师与学生可以把 AI 输出修正为可信事实后再重算。第四，边缘可观测优先。运行结果保留模型路径、推理参数、板型 schema、版本号和阶段耗时，后续可与遥测数据一起用于竞赛展示和论文实验。

第四章 后端系统设计与实现

4.1 后端目录与职责划分

LabGuardian-Server 的 app 目录按照 API、core、agent、domain、pipeline、schemas、services 和 worker 划分。api/v1 存放路由入口；core 存放配置、依赖和 Celery 应用；agent 存放 RuntimeEvidence、ContextPack、工具、验证器和状态机；domain 存放电气拓扑、板型、参考电路、比较器和风险规则；pipeline 存放从视觉到验证的阶段实现；schemas 存放请求和响应模型；services 存放业务编排；worker 用于异步任务。docs 目录则记录契约、路线图、视觉阶段协议、板型格式、错误码、边缘部署和遥测协议。

目录/文件	作用	报告中对应内容
app/pipeline/orchestrator.py	串联 S1→S1.5→S2→S3→S4→S5，管理共享模型和进度回调。	Pipeline 主流程。
app/pipeline/stages/s1_detect.py	组件检测，top 主实例，side 候选补召回。	视觉组件检测。
app/pipeline/stages/s1b_pin_detect.py	全图 YOLO-Pose 引脚检测，关联回组件。	引脚关键点识别。
app/pipeline/stages/s2_mapping.py	关键点到 hole_id/electrical_node_id 的映射和多视图投票。	孔位映射。
app/domain/circuit.py	CircuitAnalyzer、Union-Find、电气网络、SPICE/netlist 导出。	拓扑重构。
app/pipeline/stages/s4_validate.py	logical_reference_v1 比较和 validator_report_v2 输出。	验证诊断。
app/agent/context_pack.py	错误族路由、工具白名单、上下文包和指标估算。	PCM Agent。
app/agent/graph.py	ReAct + Self-Reflection 状态机与顺序 fallback。	Agent 编排。

4.2 Pipeline 编排

Pipeline Orchestrator 是后端事实链的核心。它通过线程安全单例复用 ComponentDetector 和 PinRoiDetector，避免 Celery worker 或同步请求每次都重新加载模型；同时为每次请求创建独立 BreadboardCalibrator，因为校准器包含当前图片的网格状态，不能跨请求共享。run_pipeline 接收 images_b64、reference_circuit、rail_assignments、port_annotations、net_role_assignments、net_alias_assignments、net_merge_assignments 以及 conf、iou、imgsz 等参数，依次执行检测、引脚检测、映射、拓扑、验证和语义分析，并返回 stages、total_duration_ms 与 runtime_metadata。

当前默认电源轨配置体现出运放实验场景需求：top_plus 为 VCC，top_minus 为 VEE，bot_plus 与 bot_minus 为 GND；用户可以在请求中覆盖这些设置。S3 后系统会应用端口和网络角色标注，并通过 normalize_current_netlist 添加逻辑名、别名和合并信息。S4 使用修正后的 netlist_v2 进行参考

比较。运行结束时，`runtime_metadata` 会记录代码版本、模型版本、知识库版本、规则版本、模型根目录、模型路径、设备、置信度、IoU、推理尺寸和板型参数，为后续 `benchmark` 和问题追溯提供依据。

4.3 S1：组件检测

S1 阶段输入 1 至 3 张 base64 JPEG 图片，输出 `top` 主实例和 `side` 补召回候选。代码中的策略是：`top` 视图负责建立全局 `component_id`，侧视图只负责 `supplemental_detections`，不直接进入主实例列表。这一设计比较稳妥，因为多视角下同一元件的实例融合容易引入重复或错配；先用 `top` 建立主事实，再把 `side` 作为候选证据，可以降低误合并风险。

组件检测返回 `component_detect_v1` 接口版本，包含 `detector_backend`、`detector_contract`、`detections`、`supplemental_detections`、`recall_mode`、`primary_image_shape`、解码统计和 `duration_ms`。每个 `detection` 包含 `component_id`、`component_type`、`package_type`、`pin_schema_id`、`confidence`、`bbox`、`orientation`、`view_id`、`source`、`wire_color` 和 OBB 兼容字段等。代码会过滤 `Breadboard`、`Line_area` 等背景类，并对不支持的类别显式打日志，避免“模型 raw 有输出但下游无结果”的问题被静默掩盖。对于 IC 元件，S1 还会调用封装识别逻辑推断 DIP8、DIP14 等 `package` 信息，为后续引脚生成提供依据。

4.4 S1.5：全图引脚检测

S1.5 阶段是从元件框级识别进入引脚级事实的关键。当前正式主路径不再对单个组件裁剪 ROI 后分别识别引脚，而是对 `top` 整图执行 `full-image YOLO-Pose`，再根据类别和 `bbox` 几何关系将 `pose` 实例关联回 S1 组件。这种方式能减少 ROI 裁剪误差和局部上下文丢失，也更符合未来多视角融合的证据整理需求。

`run_pin_detect` 会先判断 `top` 图像是否可用、`pin_detector` 后端是否为 `yolo_pose`、模型是否已加载。满足条件时进入 `full_image_model` 路径，输出 `component_pin_detect_v1`；否则输出 `schema-compatible` 的 `unavailable` 外壳，保证下游阶段不会因为模型缺失而结构崩溃。对于 IC 元件，代码备注说明其可以不依赖引脚模型，而是根据 `bbox` 与面包板行列约束生成 8 或 14 个引脚，再交给 S2 完成最终 `hole_id` 映射。每个 `pin` 记录 `pin_id`、`pin_name`、`keypoints_by_view`、`visibility_by_view`、`score_by_view`、`source_by_view`、`confidence`、`source` 和 `metadata`，为 S2 提供完整证据。

4.5 S2：孔位映射与多视图证据融合

S2 阶段将 S1.5 输出的 `ordered pin predictions` 映射到面包板 `hole_id` 和 `electrical_node_id`。它首先尝试用 `top` 图像对 `BreadboardCalibrator` 做校准；如果校准失败，则回退到 `synthetic grid`，并在 `calibration.mode` 中显式标记。随后，系统对每个 `pin` 构造 `observations`，根据 `keypoint`、可见性、分数、来源和投影信息生成候选孔位，调用 `BoardSchema` 将 `hole_id` 解析为 `electrical_node_id`。

S2 的一个重要特点是保留不确定性，而不是把不确定性隐藏。每个 `mapped pin` 会包含 `candidate_hole_ids`、`candidate_node_ids`、`observation_count`、`visible_view_ids`、`is_ambiguous`、`ambiguity_reasons`、`evidence_source`、`decisive_view_id`、`fusion_confidence`、`fusion_margin`、`cross_view_agreement`、`snap_distance_px` 和 `snap_confidence`。多视图融合元数据还能说明每个视图的贡献、各视图 `top1` 候选、遮挡时的权重提升和最终选择来源。

孔位吸附质量由距离和网格 `pitch` 共同决定，`snap_confidence` 使用 $1-(d/pitch)^2$ 的二次衰减公式。模型置信度高但关键点远离孔位的结果会被自动降权，低于阈值时进入 `ambiguity_reasons`。对

于两脚轴向元件、电解电容、跳线和电位器，S2 还实现了 pair selector 和几何先验，防止两个引脚被独立吸附到不符合元件物理形态的孔位。

4.6 S3：拓扑重构与 netlist_v2 生成

S3 阶段读取 S2 映射后的 components[.pins[]，通过 build_analyzer_from_components 构造 CircuitAnalyzer。CircuitAnalyzer 以 NetworkX 图表示元件与网络关系，并使用 Union-Find 合并由 Wire 连接的等电位网络。面包板静态导通关系由 BoardSchema 提供，hole_id 是 source of truth，系统会根据 hole_id 重新解析 electrical_node_id，避免上游修正 hole_id 后旧 node_id 残留导致错误。

在内部图中，元件节点和网络节点构成二分图；每条边携带 pin、role、pin_role、component、type 和 hole_id 等属性。对于 Wire 元件，系统不把它作为普通电路元件进入最终拓扑，而是通过 Union-Find 把两端网络合并。build_topology_graph 会将合并后的网络组分配为 N0、N1 等 net ID，并对非 Wire 元件建立 component-net 边。若某个非导线元件的多个引脚落入同一个网络，则会标记 same_net，用于短路风险判断。

S3 输出包括 circuit_description、netlist_v2、normalized_components、topology_graph、component_count 和 duration_ms。netlist_v2 保留 components、pins、nets、node_index 和 board_topology；其中 board_topology 含 schema_id、board_type、node_to_holes 和当前 rail_assignments，便于前端在用户拖拽或高亮时点亮完整导通条。代码还支持导出 SPICE 网表，Wire 通过 Union-Find 隐式合并，不在 SPICE 网表中单独出现。

4.7 S4：参考电路比较与诊断报告

S4 阶段只支持 logical_reference_v1 格式，不再支持孔位级或物理点位级 reference 对比。该设计符合教学场景：学生实际摆放孔位可以不同，但只要电气逻辑等价，电路应被判定为正确；相反，即使孔位看似接近，只要逻辑网络错误，就应输出诊断。S4 会把 reference_circuit 转换为 reference graph，把当前 netlist_v2 转换为 current graph，然后调用 compare_logical_graphs。

比较器的设计目标是让用户只标注输入/输出端口和电源轨，内部 signal、对称端口和无极性两脚器件顺序由系统推断。比较流程包括自动检测 reference symmetry、尝试完整图同构、失败时根据 reference 推断 current net role、判断 current 是否包含 reference 或为 reference 子图，最后才使用 graph edit distance 或 fallback 生成错接报告。匹配规则中，power/ground 严格匹配；Resistor、Capacitor、Wire 等无极性两脚器件忽略引脚顺序；LED、Diode、电解电容、三极管、电位器等功能引脚严格匹配。

S4 输出 validator_report_v2 兼容报告。每条诊断项包含 error_code、category、severity、message、suggested_action、evidence_refs、component_id、pin_name、current_hole_id、target_hole_id、current_node_id、target_node_id、expected 和 actual 等字段。常见错误码包括 FLOATING_PIN、WIRE_ENDPOINT_UNCONNECTED、COMPONENT_SHORTED_SAME_NET、POWER_RAIL_SHORT、UNEXPECTED_NET_BRIDGE、HOLE_MISMATCH、POLARITY_REVERSED、COMPONENT_MISSING、PIN_MISSING、LED_SERIES_RESISTOR_MISSING 等。前端可利用 evidence_refs 和 highlight_protocol 显示具体错误位置。

4.8 服务层与课堂状态

PipelineService 是后端业务编排的入口之一。run_sync 会根据 reference_id、inline reference 或默认配置解析参考电路，调用 run_pipeline，并把 reference 来源写入 runtime_metadata。结果构造为 PipelineResult 后，sync_result_to_classroom 会将 component_count、net_count、progress、similarity、diagnostics、comparison_report、risk_level、risk_reasons、circuit_snapshot、netlist_v2、semantic_analysis、runtime_metadata 等信息写入 ClassroomState，同时缓存缩略图。

一个值得注意的设计是 topology_label 的写入。PipelineService 会尝试调用 GNN-A 拓扑分类器，从 netlist_v2 中识别当前实验场景；只有在分类器启用、存在共识、且置信度为 high 或 medium 时，才把 topology_label 写入 station。否则返回空字符串，让 scene_resolver 进入 fail-open 状态，不会静默默认到错误场景。这是检索契约中的重要防御机制，避免 Agent 在错误场景下检索不相关 fault_case。

4.9 PCM Agent 与知识检索

当前 Agent 架构的核心思想是 PCM，即 Push-Based Context Management。它不是让大模型直接读取全部网表和全部知识库，也不是让大模型重新判断电路事实，而是先从 ClassroomState 中抽取 RuntimeEvidence，再由 context_pack.py 根据 error_code、error_tag、risk_level、current_scene_id 和用户问题构建最小上下文包。ContextPack 包含 pushed_facts、allowed_tools、prompt_rules、citation_requirements、evidence_refs 和历史摘要，并计算字符数和估算 token 数，用于控制边缘端上下文规模。

错误族路由把 COMPONENT_SHORTED_SAME_NET 映射为 short_circuit，把 NODE_MISMATCH、HOLE_MISMATCH、FLOATING_PIN 映射为 wiring_mismatch，把 POLARITY_REVERSED 映射为 polarity_error，把 LED_SERIES_RESISTOR_MISSING 映射为 missing_protection，把缺元件和缺引脚映射到 missing_component 或 incomplete_circuit。不同错误族会得到不同工具白名单：短路场景需要 netlist_trace_tool、board_schema_lookup_tool、safety_rule_lookup_tool 和 heatmap_overlay_tool；接线错误需要 board_schema 与 netlist trace；极性错误需要 datasheet_lookup_tool；概念问答可启用 teaching_concept_lookup_tool，必要时再启用 datasheet 或 circuit KB。

LangGraph 只承担编排职责。当前 graph.py 实现 classify_error → build_context_pack → react_plan → react_observe → react_reflect 的 ReAct 循环，再进入 verify_answer，验证失败则 repair_answer，最后 finalize_answer。即使没有 LangGraph 包，系统也有顺序 fallback。Verifier 要求回答包含当前错误码或 evidence_ref；如果 risk_level 为 danger，还必须包含断电、电源或短路复查提示。这种设计保障了回答的可审计性和安全性。

检索契约进一步规定 production agent graph 和蒸馏管线中合法知识源只有 teaching_scene、fault_case、datasheet_v2 和 circuit_kb，另有 station_state、pipeline_snapshot、reference_circuit 等结构化事实通道。旧的 KbService、旧 Chroma/PDF 库和 RagService.answer_with_kb 被禁止进入 Agent 主链路，避免训练部署不一致、云依赖和 wrong-scene 污染。

4.10 边缘部署与硬件遥测

edge-deployment 文档统一了模型路径和运行默认值。系统用 LABGUARDIAN_MODEL_ROOT=/app/models 作为模型根目录，组件检测和引脚检测模型分别在 component、component_detector、detect_components、pin、pin_detector、pose_components 等候选路

径下查找。默认推理尺寸统一为 960，容器环境可把 `./models` 只读挂载到 `/app/models`，并通过 `YOLO_MODEL_PATH` 与 `PIN_MODEL_PATH` 指定模型。

硬件遥测协议面向 DK-2500 运行场景，提供 `/ws/telemetry/system` 作为 5Hz WebSocket 主通道，`/api/v1/telemetry/latest` 作为 REST smoke 接口。`telemetry_frame_v1` 包含 `ts`、`cpu_pct`、`mem_used_mb`、`mem_total_mb`、`igpu_pct`、`igpu_freq_mhz`、`npu_pct`、`npu_power_mw`、`pipeline_stage` 和 `sampler_status`。采样器应能在硬件不可用或容器缺少 `sysfs` 时降级返回 `null`，不阻塞主业务。未来可将 `pipeline` 阶段耗时与 CPU/iGPU/NPU 占用曲线叠加，用于竞赛现场展示和论文实验。

第五章 前端系统设计与实现

5.1 前端总体定位

LabGuardian-Web 是 React + Vite + TypeScript 实现的单工位完整诊断演示界面。它的主要任务不是承担复杂业务规则，而是把后端结构化事实清晰呈现给学生和教师。README 中给出的演示主线为：上传面包板图片，调用 `POST /api/v1/pipeline/run`，展示 S1-S4 事实链、元件、引脚、孔位和网表，再调用 `POST /api/v1/angnt/ask`，展示 Agent 诊断解释和建议。

前端 `package.json` 显示项目使用 Vite 开发服务、TypeScript 编译、React 19、React DOM 和 lucide-react 图标库。脚本包括 `npm run dev`、`npm run build`、`npm run typecheck` 和 `npm run preview`。`src` 目录按 `api`、`types`、`components`、`features/demo` 和 `styles` 划分；`api` 封装后端接口，`types` 定义 Pipeline/Agent/UI 类型，`components` 实现上传、画布标注、阶段耗时、诊断、网表和原始 JSON 展示，`features/demo` 则组织 reducer、hooks 和主流程。

5.2 API 封装

`src/api/client.ts` 提供统一 `requestJson`。它从 `VITE_API_BASE_URL` 读取后端地址，默认请求超时 5 分钟，使用 `AbortController` 控制超时，并在 `response.ok` 为 `false` 时解析 `detail` 字段生成 `ApiError`。这个封装保证了 Pipeline 推理较慢时前端不会过早失败，也能给用户显示“请求超时，请确认后端服务和模型推理状态”的友好错误。

`src/api/pipeline.ts` 封装 `getHealth`、`getVersion`、`runPipeline`、`recomputeCorrected`、`analyzeCircuit` 和 `visualizePorts`。`src/api/agent.ts` 封装 `askAgent` 与 `getAgentStatus`，并提供 `waitForAgentResult` 轮询逻辑，默认每 900ms 轮询一次，最多 120 次。前端 README 中列出的后端契约包括 `GET /health`、`GET /version`、`POST /api/v1/pipeline/run`、`POST /api/v1/angnt/ask` 和 `GET /api/v1/angnt/status/{job_id}`。

5.3 DemoPage 主页面

DemoPage 是前端演示的中心。它使用 `demoReducer` 管理状态，通过 `useBackendStatus` 检查后端在线状态，通过 `useReferences` 获取参考电路，通过 `usePipelineRun` 执行 Pipeline，通过 `useAgentChat` 处理 Agent 对话，通过 `useTopologySuggestion` 在生成 `netlist_v2` 后自动获取 GNN-A 拓扑建议。页面顶层包括 `AppHeader`、`MetricStrip`、`UploadPanel`、`ReferenceSelector`、`Evidence Chain` 主区域、`DiagnosticsPanel`、`AgentChat`、`StageTimeline` 和 `RawJsonPanel`。

主区域有两种展示方式：当 `activeMode` 为 `netlist` 时，显示 `NetlistView`，支持孔位修正、端口标注、网络角色、引脚极性和 IC 标注；否则显示 `ResultCanvas`，在原始图片上渲染识别框和高亮目标。

DiagnosticsPanel 按严重程度对 comparison_report.items 排序，用户选择某条诊断后，DemoPage 会从 evidence_refs 中提取 highlightTargets 传给 ResultCanvas 或 NetlistView，实现“点击错误—定位元件/引脚/孔位”的交互。

5.4 上传、运行与阶段进度

上传流程中，handleFileSelected 会检查文件类型是否为 image/*，再通过 fileToBase64 转换为 base64 字符串，保存到状态中。usePipelineRun 的 execute 会调用 runPipeline，提交 station_id、images_b64、conf、iou、imgsz、reference_id、rail_assignments、port_annotations、net_role_assignments 和 ic_annotations。运行成功后，前端保存 PipelineResult，并自动调用 Agent 生成演示用诊断解释。

为了提升演示体验，usePipelineRun 定义了 STAGE_RHYTHM，将 detect、pin_detect、mapping、topology、validate、semantic_analysis 分别设置为 1200ms、1800ms、1000ms、800ms、600ms 和 400ms 的前端进度节奏。这不是后端真实耗时统计，而是用于在请求过程中展示阶段推进。真实耗时仍应以后端 stages[].duration_ms、total_duration_ms 和 runtime_metadata 为准。

5.5 人工修正与重算

人工修正是前端的重要工程能力。handleApplyCorrections 会从当前 PipelineResult 的 mapping 阶段取出 components，结合 manualCorrections 生成 correction patch，再整理 portAnnotations、manualNetRoleAssignments、manualIcAnnotations 和 pinPolarityAssignments。如果没有任何修正或标注，前端会提示用户先标注端口、修改孔位或指定网络角色。若存在有效修改，则调用 recomputeCorrected，把这些结构化输入提交给后端。

这种重算机制让系统从“纯自动 AI 演示”转向“可纠错的教学工具”。在真实课堂中，视觉模型难免出现漏检、误检或孔位吸附偏差。如果系统不能允许人工修正，教师就只能放弃这次结果；而 LabGuardian 允许保留视觉阶段识别到的大部分事实，只对有争议的 pin 或端口做局部修改，再让拓扑和比较模块重新判断，这更符合实验教学中的人机协同逻辑。

第六章 数据结构与接口设计

6.1 主要接口

接口	方法	用途
/health	GET	前端启动时检查后端服务是否在线。
/version	GET	获取代码版本、模型版本、规则版本等展示信息。
/api/v1/pipeline/run	POST	同步执行完整 Pipeline，演示场景直接返回 PipelineResult。
/api/v1/pipeline/submit	POST	异步提交 Pipeline 任务，需要 Celery 和 Redis。
/api/v1/pipeline/status/{job_id}	GET	查询异步 Pipeline 任务状态。

/api/v1/pipeline/recompute-corrected	POST	应用前端人工修正后重跑 S3/S4/S5。
/api/v1/pipeline/analyze	POST	返回元件引脚定位、网表和拓扑图。
/api/v1/pipeline/compare-netlist	POST	调试接口，直接比较 reference 与 current_netlist_v2。
/api/v1/pipeline/visualize/ports	POST	返回端口映射列表，用于前端可视化。
/api/v1/angnt/ask	POST	提交 Agent 问答任务。
/api/v1/angnt/status/{job_id}	GET	轮询 Agent 结果。
/ws/telemetry/system	WebSocket	实时推送硬件遥测帧。

6.2 PipelineRequest 与 PipelineResult

PipelineRequest 的核心字段包括 station_id、images_b64、conf、iou、imgsz、reference_id、reference_circuit、rail_assignments、port_annotations、net_role_assignments、net_alias_assignments、net_merge_assignments、pin_polarity_assignments 和 ic_annotations。前端当前将图片转为不含 data:image 前缀的纯 base64 字符串，便于后端统一解码。rail_assignments 用于指定电源轨角色；port_annotations 只要求用户标注输入/输出端口；net_role_assignments、alias 和 merge 主要用于高级调试或人工修正。

PipelineResult 是前端、课堂状态和 Agent 共用的结果模型。它承载 job_id、station_id、stages、component_count、net_count、progress、similarity、diagnostics、risk_level、risk_reasons、comparison_report、runtime_metadata 等字段。stages 中的每个阶段都包含 stage 名称、数据和耗时；runtime_metadata 则记录模型路径、推理参数、参考电路来源、人工标注和网络归一化信息。

6.3 netlist_v2

netlist_v2 是当前主链路最重要的数据结构。它不是简单的文字网表，而是一个可被前端、validator、Agent 和后续实验统计共同消费的结构化对象。其核心内容包括 board_schema_id、components、nets、node_index 和 board_topology。components 中每个元件包含 component_id、component_type、package_type、part_subtype、polarity、orientation、symmetry_group、pins、confidence 和 metadata；pins 中保留 pin_id、pin_name、hole_id、electrical_node_id、electrical_net_id、observations、confidence、is_ambiguous 和 metadata。

nets 表示由 Union-Find 合并后的电气网络，每个 ElectricalNet 包含 electrical_net_id、member_node_ids、member_hole_ids 和 power_role。node_index 保存 electrical_node_id 到 hole_id 的索引，board_topology 保存完整的 node_to_holes 和 rail_assignments。这样，前端不仅能知道某个引脚属于哪个 net，还能在用户点击某个网络时高亮全部相关孔位。

6.4 validator_report_v2 与高亮协议

validator_report_v2 是诊断结果的核心报告格式。summary 中包含 total_item_count、logic_correct、similarity、comparison_mode、match_type、ignore_component_id、ignore_hole_id、

ignore_passive_pin_order、allow_extra_wires、strict_functional_pin_roles 和 report_layers 等信息。items 则是具体诊断项，每项携带 error_code、category、severity、message、suggested_action 和 evidence_refs。

高亮协议 labguardian_highlight_v1 把 evidence_refs 转换为前端可直接绘制的 targets。target 可以是 component_bbox_ref，渲染为元件框；可以是 pin_keypoint_ref，渲染为引脚点；也可以是 hole_candidate_ref，渲染为当前孔位、目标孔位和候选孔位。AgentService 也可以把同一份高亮协议作为 evidence 输出，使自然语言解释与可视化定位保持一致。

6.5 RuntimeEvidence 与 ContextPack

RuntimeEvidence 是 Agent 从 ClassroomState 中抽取的运行证据，包含 station_id、risk_level、error_codes、error_tags、findings、netlist_v2、circuit_snapshot、reference_circuit、current_scene_id、history_facts 和 history_summary 等。它是 Agent 的事实输入，而不是自然语言长上下文。ContextPack 则是 PCM 编译后的每轮上下文，包含 pushed_facts、allowed_tools、prompt_rules、citation_requirements 和 evidence_refs，并附带 ContextPackMetrics 记录推送事实数、工具数、证据数、历史长度和估算 token。

这样的结构使 Agent 的行为可控：它只能看到被推送的事实和被允许的工具，不会越过 validator 重新猜测孔位或网络。对于边缘端部署，这也有助于减少上下文长度，降低小模型或规则模板的负担。

第七章 关键技术与创新点

7.1 结构化事实链

项目最大的工程创新在于把“看图诊断电路”拆成多个可解释的事实转换步骤，而不是用单个端到端大模型直接回答。S1 负责元件实例，S1.5 负责引脚候选，S2 负责孔位吸附，S3 负责导通网络，S4 负责逻辑比较，Agent 负责解释。这种结构化事实链让每一步都能被单独测试、单独可视化、单独修正，也使错误定位更具体。

7.2 多视角与抗遮挡策略

项目方案强调多角度拍摄解决遮挡问题。当前代码中，S1 已支持 side recall candidates，S2 的协议已经包含多视图 observation、evidence_source、decisive_view_id、fusion_confidence、fusion_margin 和 cross_view_agreement。虽然前端当前默认单图演示，但后端数据结构和 S2 融合逻辑已经为多视角扩展预留空间。特别是遮挡感知权重规则会在 top 不可见或置信度低时提高 side 视图权重，避免单一俯视图失效。

7.3 BoardSchema 与 Snap-to-Grid

BoardSchema 把物理孔位规范化为稳定 electrical_node_id，是连接视觉与电气逻辑的桥梁。默认比赛板 schema 覆盖 A1-E63、F1-J63 和 LP/LN/RP/RN 两段电源轨，并兼容历史 rail_top+、rail_top-、rail_bot+、rail_bot- 命名。Snap-to-Grid 不仅选取最近孔位，还结合 snap_distance、pitch、候选列表、元件几何约束和多视图投票形成可信度评估。

7.4 图论拓扑与逻辑比较

CircuitAnalyzer 使用 NetworkX 和 Union-Find 生成电气拓扑，compare_logical_graphs 则把参考电路和当前网表都转换为 component-net 二分图，再进行拓扑感知比较。相比逐孔位对比，图比较可以容忍无极性器件的引脚互换、内部信号名称差异和合理的额外导线；相比纯文本规则，它又能严格处理电源、地、功能引脚和极性器件。

7.5 PCM Agent 与可验证回答

PCM Agent 的创新在于“先压缩事实，再选择工具，最后生成回答”。它根据错误族和意图动态选择工具白名单，避免把所有技能、知识和历史都塞进上下文。Verifier 则保证输出必须引用错误码或证据，并在危险风险时输出安全提醒。该机制特别适合实验教学：学生得到的是可追溯的解释，教师也可以根据 evidence_refs 检查系统是否真的基于当前电路作答。

7.6 人工修正闭环

许多 AI 演示系统只关注“模型一次性给出答案”，但真实教学现场需要可修正。LabGuardian 的 recompute-corrected 接口和前端 NetlistView 让用户把 AI 识别结果修成可信事实，再让后端重新执行拓扑和验证。这种闭环把 AI 作为助教而非裁判，既增强可用性，也符合工程教育中“学生理解并修正错误”的目标。

7.7 边缘可观测与可复现实验

项目不仅关注功能，还关注运行可观测性。runtime_metadata 记录模型路径、版本、参数、设备和板型；telemetry_frame_v1 记录 CPU、内存、iGPU、NPU 和 pipeline_stage；docs 规定改协议必须同步文档和测试。对竞赛和论文而言，这些机制可以支撑 p50/p90 延迟、峰值内存、模型版本、上下文长度、工具调用数和验证结果等实验指标。

第八章 测试验证与质量保障

8.1 现有测试线索

仓库 README 和 roadmap 显示，项目已经补充最小 regression fixture 与 smoke tests，并提供多类验证命令。后端可运行 board schema 默认映射冒烟测试、逻辑图比较与 S4 回归测试、DSL 参考电路加载与编译测试、pipeline 合同与阶段级回归测试以及全量 pytest。开发路线中还提到多视图融合、孔洞吸附质量、Agent ReAct 循环、硬件遥测和 VLM provider contract 等测试。

8.2 建议的测试体系

测试层级	测试对象	重点指标
单元测试	BoardSchema、Union-Find、net normalization、error code、ContextPack	映射正确性、边界输入、空值降级、错误码稳定。

阶段测试	S1、S1.5、S2、S3、S4、S5	阶段输入输出 schema、duration_ms、fallback 标记、不确定性字段。
回归测试	典型参考电路和错接 fixture	逻辑正确率、错误码一致性、suggested_action 稳定性。
前端测试	上传、结果展示、人工修正、Agent 轮询	状态流转、错误提示、按钮禁用、高亮目标一致性。
集成测试	前端→后端→Agent 端到端	完整链路可运行、PipelineResult 和 AgentResult 可解析。
边缘测试	DK-2500 运行、模型路径、遥测、延迟	p50/p90、峰值 RSS、CPU/iGPU/NPU 占用、降级行为。

8.3 数据集与评测建议

视觉模型评测应将元件检测、引脚检测和孔位映射分开统计。元件检测可统计 mAP、漏检率和误检率；引脚检测可统计关键点误差、可见性分类和 ordered pin 顺序正确率；孔位映射应统计 Top-1 hole 准确率、Top-k 候选覆盖率、snap_confidence 分布和低置信场景召回率。对于多视图，建议比较 top-only 与 multi-view 的 A/B 结果，重点关注遮挡、反光、强弱光、倾斜拍摄和复杂布线样本。

拓扑和诊断评测应以电路级任务为单位。可选择一阶 RC、反相运放、同相运放、NE555 定时器、LED 限流、三极管开关等教学模板，构造正确、缺线、错孔、短路、极性反接、缺保护电阻、端口标注错误等场景。指标包括逻辑正确判定准确率、错误码命中率、错误定位准确率、建议动作可执行性、误报率和漏报率。

8.4 Agent 质量保障

Agent 测试不应只看自然语言是否“像人话”，还要检查是否遵守证据约束。建议为每个 error family 构造 golden case，验证 ContextPack.allowed_tools 是否符合预期，工具调用顺序是否在白名单内，回答是否包含至少一个当前错误码或 evidence_ref，danger 场景是否优先提醒断电与短路复查。对于概念问答、实验指导和混合意图，应检查检索源是否符合 retrieval contract，wrong-scene rate 和 legacy fallback rate 必须为零。

第九章 部署运行与边缘化方案

9.1 本地开发部署

后端快速开始流程为安装开发依赖、启动 Redis、启动 Celery Worker、启动 FastAPI。演示阶段可以直接使用同步接口 POST /api/v1/pipeline/run，无需异步任务；如果要测试 /submit 和 /status/{job_id}，则需要 Redis 和 Celery Worker。前端启动流程为安装 npm 依赖后运行开发服务，若后端不在 127.0.0.1:8000，则通过 VITE_API_BASE_URL 指定后端地址。

9.2 Docker 与模型目录

边缘部署建议采用统一模型根目录 `/app/models`，并把模型文件以只读方式挂载到容器。组件检测模型和引脚检测模型分别配置 `YOLO_MODEL_PATH` 和 `PIN_MODEL_PATH`；若显式路径不存在，服务会按候选路径查找，再回退到仓库内 `train_demo` 权重。为了保证复现实验，模型版本、推理尺寸、置信度阈值和板型 `schema` 应写入 `runtime_metadata`。

9.3 DK-2500 部署设想

根据初选方案，DK-2500 搭载 Intel Core Ultra 5 225U，适合展示 CPU、iGPU、NPU 异构协同。当前代码层面已经具备设备配置、`runtime_metadata`、`telemetry` 和 OpenVINO/GenAI optional dependency 等准备，但 S1/S1.5 的 ONNX 导出、OpenVINO GPU/NPU 插件适配、INT8 量化、PT vs ONNX parity test、真机 p50/p90 延迟和峰值内存等仍属于后续 edge benchmark 工作。

9.4 运行安全与降级策略

实验教学系统必须有降级策略。视觉模型缺失时，S1.5 可以输出 `unavailable` 外壳；校准失败时，S2 使用 `synthetic_fallback` 并标记低可信；拓扑分类器失败时，`scene_id` 为空并跳过 `fault_case` 检索；硬件遥测字段不可用时返回 `null`；Agent 没有真实 LLM 时仍使用确定性模板 `provider`。这样的 `fail-open` 或 `fail-closed` 策略能避免系统在现场因一个模块不可用而整体崩溃，同时也防止错误默认值污染诊断。

第十章 项目进度、现状与差异说明

10.1 当前已形成的工程成果

- 后端服务骨架已建立，包括 `pipeline_service`、`guidance_service`、`version_service`、`rag_service`、`agent_service` 和 `classroom_state`。
- 新网表模型 `netlist_v2` 已成为主输出结构，S3/S4/validator 和前端均围绕该结构消费事实。
- 视觉主链已固定为 YOLO-Detect 组件检测和 full-image YOLO-Pose 引脚检测，OBB 和 ROI fallback 保留兼容但不作为默认主路。
- S2 已输出 `components[].pins[]`、候选孔位、候选节点、多视图证据、不确定性和吸附质量字段。
- BoardSchema 默认比赛板已支持 63 行主区和分段电源轨，用户只需标注电源轨和输入/输出端口。
- S4 已基于 `logical_reference_v1` 和 topology-aware graph compare 输出 `validator_report_v2`。
- PCM Agent 已具备 RuntimeEvidence、ContextPack、错误族路由、确定性工具、ReAct 循环、Verifier 和 repair 分支。
- Web 前端已完成单工位演示界面，支持上传、事实链展示、诊断卡片、Agent 对话和人工重算。
- 文档体系较完整，覆盖视觉契约、比较架构、板型格式、错误码、检索契约、边缘部署和遥测协议。

10.2 与初选方案的差异

初选方案从竞赛表达角度提出了更完整的“面包板原型验证 → PCB 成品制造”全周期闭环，并强调 NPU 上的 VLM 微观诊断和 Anomalib 热力图。但当前 GitHub 主仓库更聚焦面包板电路结构化

诊断，README 中明确说明 PCB/AOI 相关代码已移除，不应重新引入平行子系统；agent/graph.py 中也说明 vlm_explain 分支因项目范围和检索隔离问题从图中删除。

这种差异不一定是缺点。相反，它表明团队在工程实现中进行了收敛：先把最核心、最可验证、最能支撑教学闭环的“图像→孔位→网表→比较→解释”主链路打通，再把 VLM、PCB AOI 和更复杂的边缘异构加速作为未来扩展。对于正式项目报告，建议以当前代码实现为主，保留初选方案中的宏观背景和未来展望，避免把尚未纳入主链路的能力描述为已完成。

10.3 阶段性计划

阶段	目标	验收产物
近期	稳定 Pipeline 契约和前端演示，补齐关键错接场景。	更多 fixture、回归测试、前端高亮和人工修正体验优化。
中期	完成 DK-2500 真机部署与 edge benchmark。	ONNX/OpenVINO 模型、INT8 量化实验、p50/p90/RSS/telemetry 报告。
中期	扩展 reference DSL 和教学知识库。	更多教学模板、datasheet_v2、fault_case 和 circuit_kb 条目。
后期	引入更强的多视角融合和低置信交互。	遮挡样本 A/B 测试、可视化证据 overlay、标定误差统计。
扩展	评估 VLM/PCB AOI 是否重新纳入独立模块。	独立 scope、独立 API、独立测试，不破坏主链路检索契约。

第十一章 风险分析与改进计划

11.1 视觉识别风险

视觉模型的主要风险是遮挡、反光、拍摄角度、元件类别不平衡和引脚关键点误差。改进方案包括继续扩充真实实验样本，强化遮挡、强弱光和密集布线数据；对低 snap_confidence 和 multi_view_vote_conflict 的样本建立人工复核机制；将 decisive_view_id、per_view_contribution 和 snap_distance 可视化到前端；对 IC、轴向元件和电位器等特殊元件继续完善几何先验。

11.2 拓扑与参考电路风险

拓扑比较依赖 reference DSL 和端口标注。如果参考电路定义不准确、端口标注缺失或电源轨设置错误，系统可能给出错误诊断。改进方案包括为 reference DSL 建立审查流程，为每个实验模板提供示意图和默认端口提示，在前端运行前强制检查 rail_assignments 和必要端口，提供 compare-netlist 调试接口帮助教师验证参考电路。

11.3 Agent 与知识检索风险

Agent 的风险主要是 wrong-scene 检索、旧知识源污染、自由发挥和安全提示缺失。当前 retrieval contract 已经通过合法源清单、禁用源清单、训练部署不变量和 fail-closed 机制降低风险。后续应继

续执行 gold set 评测，监控 wrong-scene rate、legacy fallback rate、datasheet recall 和回答证据覆盖率；新增任何检索入口都必须更新 retrieval-contract.md 并补测试。

11.4 边缘部署风险

边缘部署风险包括模型体积、推理延迟、共享内存压力、OpenVINO 设备兼容、NPU 驱动路径不稳定和容器权限限制。改进方案包括拆分 server-dev、edge-cpu、edge-openvino 镜像；导出 S1/S1.5 ONNX 并做 parity test；记录阶段 latency、p50/p90、peak RSS、模型版本和 board_schema_id；对 telemetry sampler 采用 defensive 设计，避免硬件采样异常影响主服务。

11.5 前端体验风险

前端需要避免把复杂 JSON 直接暴露给学生。建议将 NetlistView 的高级修正入口分层隐藏，普通学生优先看到“哪里错、为什么、怎么改”；教师或开发者再展开 candidate holes、evidence_refs、raw JSON 和 manual net roles。对于错误卡片，应提供一键高亮、修复步骤、参考图示和安全提醒；对于低置信结果，应明确提示“需人工确认”，避免误导。

第十二章 应用价值、推广路径与实施组织

12.1 对学生学习过程的价值

LabGuardian 对学生最大的价值是把实验反馈从“等待教师巡查”转变为“即时自助排错”。在传统课堂中，学生经常在某个接线点卡住十几分钟，最终得到的反馈可能只是教师一句“这里接错了”。这种反馈虽然能解决眼前问题，却不一定帮助学生理解面包板导通关系、元件引脚功能和电路拓扑。LabGuardian 的诊断结果包含元件、引脚、孔位、节点、网络、参考连接和建议动作，学生可以沿着证据链理解错误产生原因，从而把一次排错转化为一次概念强化。

系统还可以支持形成性评价。学生每次上传和修正都会产生结构化数据，例如常见错误码、错孔类型、短路风险、端口标注错误、低置信孔位和修复前后相似度变化。这些数据可以帮助学生回顾自己的薄弱环节，也可以作为实验报告中的过程性证据。与只看最终波形是否正确相比，过程数据更能反映学生是否真正理解了电路搭建逻辑。

12.2 对教师教学组织的价值

对教师和助教而言，LabGuardian 可以把大量重复检查工作前置给系统。教师不再需要逐个实验台检查“电阻是否跨行”“LED 是否有限流电阻”“电源轨是否短路”“运放正负电源是否接反”等基础问题，而是可以优先处理系统标记为 danger 或 warning 的工位。课堂状态中保存的 risk_level、diagnostics、comparison_report、netlist_v2 和 topology_label 可以进一步汇总为教师看板，显示哪些实验模板最容易出错、哪类元件最常被误接、哪些学生需要重点辅导。

这种看板并不是为了替代教师，而是帮助教师把注意力从机械巡检转向概念讲解和个性化指导。当某一错误码在多个工位频繁出现时，教师可以暂停全班演示，统一讲解对应知识点；当某个工位多次出现低 snap_confidence 或多视图冲突时，教师可以指导学生调整拍摄角度或复查遮挡区域。

12.3 对课程资源建设的价值

LabGuardian 的 reference DSL、logical_reference_v1、teaching_scene、fault_case、datasheet_v2 和 circuit_kb 可以逐步沉淀为课程知识资产。每个实验模板不仅包含电路原理，还包含参考拓扑、端口语义、预期测量点、常见故障、错误码解释和安全提醒。随着课程迭代，教师可以不断补充新的参考电路和故障案例，使系统从单次演示工具发展为电子实验课程的数字化资源库。

这类资源库具有可复用性。不同学校的电子基础实验虽然教材和器件略有差异，但面包板导通规则、两脚无极性元件、LED 限流、运放供电、三极管引脚等知识具有通用性。只要 BoardSchema、参考 DSL 和 datasheet 条目支持版本化，系统就能迁移到不同课程、不同实验箱和不同板型。

12.4 项目实施组织建议

从工程实施角度，团队可按照“视觉算法、后端架构、前端交互、知识库与测试、边缘部署”五条线并行推进。视觉算法线负责数据采集、标注规范、YOLO-Detect/YOLO-Pose 训练、孔位吸附误差分析和多视图融合；后端架构线负责 Pipeline 契约、BoardSchema、netlist_v2、validator 和服务层接口；前端交互线负责上传、证据链展示、人工修正、Agent 对话和教师看板；知识库与测试线负责 reference DSL、fault_case、datasheet_v2、golden tests 和检索契约；边缘部署线负责 DK-2500 环境、模型转换、OpenVINO、telemetry 和 benchmark。

协作时应坚持“先改契约，再改代码，再补测试，再更新文档”。例如修改 S2 输出字段时，必须同步更新 vision-stage-contracts.md 和 tests/pipeline；修改错误码时，必须同步更新 validator-error-codes.md 和 fixture；修改 Agent 检索行为时，必须同步更新 retrieval-contract.md，并通过 wrong-scene 与 legacy fallback 检查。这样可以降低多人并行开发时的数据结构漂移风险。

12.5 推广路径与落地成本

推广路径可以分为三个阶段。第一阶段是单工位演示，目标是跑通一套典型实验，如 RC 电路、LED 限流或运放反相放大器，展示上传图片、生成网表、发现错误和 Agent 解释。第二阶段是课程实验室试点，目标是在若干实验台部署边缘服务器或共享服务器，采集真实错接样本，优化视觉模型和前端交互。第三阶段是课程资源平台化，目标是形成参考电路库、故障案例库、教学概念库和教师看板，把系统嵌入实验教学流程。

落地成本主要包括边缘计算设备、摄像设备或学生手机、模型训练数据、教师维护参考电路的时间，以及系统部署维护成本。当前设计不依赖固定工业相机，优先支持学生手机上传，这有利于降低硬件门槛。服务器端若能在 DK-2500 本地运行，则可以减少云资源费用和网络依赖。长期看，最大的成本不是硬件，而是高质量数据和课程知识资产建设，因此项目应重视数据标注规范、知识条目版本化和教师可编辑工具。

12.6 阶段性交付清单

交付类别	交付内容	验收标准
功能交付	图像上传、Pipeline 运行、网表展示、诊断卡片、Agent 解释、人工重算。	至少一套典型实验可稳定端到端演示。

数据交付	参考电路、错接样本、datasheet JSON、fault_case、测试 fixture。	每个模板覆盖正确与典型错误场景。
算法交付	组件检测、引脚检测、孔位映射、拓扑比较、风险分类。	输出字段符合契约，错误码和证据可追溯。
部署交付	后端启动脚本、前端启动脚本、模型目录、Docker/Redis/Celery 配置。	新环境能按文档复现演示。
文档交付	项目报告、接口说明、测试说明、部署说明、答辩材料。	文档与当前代码一致，不夸大未纳入主链路的能力。

12.7 典型课堂演示流程

为了让项目展示更清晰，建议把现场演示设计为“正常电路—错误电路—人工修正—Agent 解释—边缘遥测”五步。第一步展示一个正确搭建的参考电路，上传图片后让系统输出较高相似度和无严重错误报告，说明系统能理解正确拓扑。第二步将某个电阻或导线移动到错误孔位，重新上传后展示 HOLE_MISMATCH、NODE_MISMATCH 或 COMPONENT_SHORTED_SAME_NET 等错误码，并点击诊断卡片让前端高亮具体引脚和孔位。第三步在 NetlistView 中人工修正低置信孔位或补充端口标注，调用 `recompute-corrected`，说明系统支持人机协同而不是一次性黑盒判断。第四步打开 AgentChat，让系统用自然语言解释错误原因、证据和修复步骤，强调回答引用的是 `validator_report_v2` 与 `netlist_v2` 事实。第五步展示 `runtime_metadata` 或 `telemetry`，说明系统为 DK-2500 边缘部署和异构计算展示预留了可观测数据。

答辩时应避免把所有技术一次性堆给评委，而应围绕“为什么难、我们如何拆解、当前做到了什么、下一步如何验证”展开。可以先用一张面包板实物图解释遮挡和孔位问题，再用一张数据流图解释 S1 到 S4 事实链，然后展示前端界面中的 `evidence chain`，最后说明 Agent 为什么不会凭空猜测。对于初选方案中的 PCB AOI 和 VLM 微观诊断，可以坦诚说明它们属于后续扩展，当前主线已经根据代码收敛到面包板拓扑诊断，工程边界更清晰、验收目标更可控。这样的表达比单纯宣称“系统使用大模型”更能体现工程成熟度。

在课堂试点中，演示脚本还可以转换为学生操作流程：学生先按实验指导书搭建电路，使用手机拍摄上传；系统返回风险等级和错误位置；学生依据建议修改电路；修改后再次上传验证；最终将诊断前后对比、修复说明和测量结果写入实验报告。教师则可以在后台查看班级错误统计，选择共性问题进行集中讲解。这种流程把 LabGuardian 嵌入已有教学环节，而不是要求课程完全重构。

第十三章 结论

LabGuardian 选题具有明确的教学价值和工程挑战。它面向高校电子实验中真实存在的痛点：教师巡检压力大、学生排错慢、面包板遮挡严重、二维原理图到三维实物映射困难、诊断结果缺少证据链。项目并没有停留在“用 AI 看一张图片”的层面，而是把视觉识别、孔位映射、板型知识、图论拓扑、参考电路比较、诊断报告、前端高亮和 Agent 解释串联成一条完整事实链。

从当前代码看，项目最扎实的部分是后端结构化 Pipeline 和数据契约。S1/S1.5/S2 把图像转为 pin/hole 事实，S3 通过 BoardSchema 和 Union-Find 生成 `netlist_v2`，S4 通过 `topology-aware graph`

compare 生成 validator_report_v2, Agent 通过 RuntimeEvidence 和 ContextPack 在证据约束下回答问题。前端则提供了足够完整的演示闭环, 支持上传、展示、诊断、对话和人工修正。

后续工作的重点应是以真实样本和真机数据验证系统可靠性, 而不是继续扩大概念范围。建议优先完成多视角样本评测、错接 fixture 扩充、前端高亮完善、DK-2500 edge benchmark 和知识库 gold set 评估。在此基础上, 再评估是否将初选方案中的 VLM 微观诊断和 PCB AOI 作为独立扩展模块重新纳入。总体而言, LabGuardian 已具备从竞赛演示走向教学工具原型的基础, 若能持续加强数据、测试和边缘部署验证, 将有潜力成为高校电子实验智能化教学的有价值样板。

参考资料

1. 《LabGuardian: 基于边缘 AI 的智能实验助教系统》初选项目设计方案书。
2. LabGuardian-Server 仓库 README、docs/README.md、comparison-architecture.md、vision-stage-contracts.md、board-schema-format.md、validator-error-codes.md、pcm-agent-architecture.md、retrieval-contract.md、edge-deployment.md、telemetry-protocol.md。
3. LabGuardian-Server 核心源码: app/pipeline/orchestrator.py、s1_detect.py、s1b_pin_detect.py、s2_mapping.py、s3_topology.py、s4_validate.py、app/domain/circuit.py、app/services/pipeline_service.py、app/agent/context_pack.py、app/agent/graph.py。
4. LabGuardian-Web 仓库 README、package.json、src/api/client.ts、src/api/pipeline.ts、src/api/agent.ts、src/features/demo/DemoPage.tsx、src/features/demo/usePipelineRun.ts。

附录：术语与文件索引

术语	含义
hole_id	面包板物理孔位编号, 如 A12、LP31。
electrical_node_id	面包板静态导通节点编号, 由 BoardSchema 根据 hole_id 解析。
electrical_net_id	动态电气网络编号, 由元件和导线连接合并后生成。
netlist_v2	当前主链路的结构化网表格式, 包含元件、引脚、网络和板型拓扑。
logical_reference_v1	教师或参考模板提供的逻辑参考电路格式。
validator_report_v2	S4 输出的结构化诊断报告, 包含错误码、证据和建议。
RuntimeEvidence	Agent 从课堂状态抽取的当前运行证据。
ContextPack	PCM 编译后的最小上下文包, 包含事实、工具、规则和证据引用。
BoardSchema	描述面包板孔位、导通节点、电源轨和别名的板型规则。
Snap-to-Grid	将视觉关键点吸附到最近面包板孔位并计算置信度的过程。

文件	建议阅读目的
README.md	快速了解后端当前主链路、目录结构和运行命令。
docs/vision-stage-contracts.md	开发或修改 S1/S1.5/S2 协议前必读。
docs/comparison-architecture.md	理解 reference DSL、netlist normalization 和 graph compare。
docs/retrieval-contract.md	修改 Agent/RAG/蒸馏检索路径前必读。
docs/validator-error-codes.md	维护错误码、前端高亮和诊断报告格式。
docs/edge-deployment.md	部署模型路径、runtime_metadata 和边缘 benchmark。
LabGuardian-Web/src/features/demo/DemoPage.tsx	理解前端演示页面状态、布局和人工修正流程。